

Rapport technique — Projet Déneiger Montréal

Auteurs

Amokrane Ait-Taouit

Imran Belmessaoud

Emmanueli-David Valle Mvoula-Moukouyou

Amar Kaci

Christina Gapy

1. Introduction

La ville de Montréal consacre chaque année des ressources importantes aux opérations de déneigement afin de maintenir la circulation, l'accès aux services essentiels et la continuité des activités économiques et sociales. Dans ce projet, nous nous intéressons plus particulièrement au déblaiement du réseau routier lors d'épisodes neigeux modérés.

L'objectif est de proposer une méthode permettant d'organiser efficacement les tournées de déneigeuses sur plusieurs secteurs de Montréal, tout en respectant le code de la route, en minimisant les coûts et en comparant plusieurs politiques de priorisation.

Pour cela, le projet s'appuie sur :

- une modélisation en graphe du réseau routier ;
- des données OpenStreetMap ;
- un solveur d'optimisation exacte ;
- un ensemble d'outils d'évaluation.

2. Organisation du projet

Le dépôt est structuré en plusieurs modules spécialisés :

- **snowplow_routing.py** : résolution du problème de tournée ;
- **priorities.py** : définition des scénarios de priorisation ;
- **scenarios.py** : orchestration complète des calculs ;
- **cost_model.py** : modèle économique ;
- **demo.py** : script de démonstration ;
- **app.py** : interface Streamlit ;
- **README.md, THEORY.md, Limits.md** : documentation ;
- **tests/** : tests unitaires.

Cette séparation est importante, car elle permet de distinguer clairement :

- la partie mathématique ;
- la partie métier liée à la priorisation ;
- la partie économique ;
- la partie visualisation.

3. Formalisation du problème

Le problème traité n'est pas un simple problème de plus court chemin. Il s'agit d'un problème de couverture de rues : la déneigeuse doit parcourir chaque segment requis au moins une fois.

Le réseau est modélisé comme un graphe orienté :

- les sommets représentent les intersections ;
- les arcs représentent les rues ;
- une rue à double sens est modélisée par deux arcs opposés ;
- une rue à sens unique n'est représentée que dans son sens légal.

Cette modélisation est cohérente avec le contexte réel, car elle garantit que les tournées calculées respectent les contraintes de circulation.

Le problème est donc proche :

- du problème du postier chinois ;

- et, dans les phases de priorisation, du problème du postier rural.

4. Pourquoi avoir choisi cette approche algorithmique

Nous avons choisi une approche de programmation linéaire en nombres entiers avec OR-Tools, complétée par l'algorithme de Hierholzer pour reconstruire les tournées.

Ce choix est pertinent pour plusieurs raisons :

- le problème porte sur la couverture d'arcs, et non sur la simple visite de nœuds ;
- la prise en compte des rues à sens unique est naturelle dans un graphe orienté ;
- un solveur exact permet d'obtenir une solution optimale sur le modèle choisi ;
- le modèle est réutilisable pour tous les secteurs ;
- la priorisation peut être intégrée sans changer le cœur de l'algorithme.

5. Description détaillée du code

5.1 Le module snowplow_routing.py

Ce fichier constitue le cœur algorithmique du projet. Il contient :

- le calcul du coût des segments ;
- la résolution par optimisation ;
- la reconstruction de la tournée ;
- la conversion des données OpenStreetMap.

5.1.1 Calcul du coût d'un segment

La fonction **segment_cost** transforme la longueur d'une rue en coût :

```
def segment_cost(length_m):
    """Cost of a single street segment from its length in metres.

    A missing OSM `length` defaults to 0 (near-universal on `drive` networks)."""
    length_km = length_m / 1000.0
    time_h = length_km / SPEED_KMH
```

Cette fonction est importante car elle permet de convertir une donnée géographique brute en grandeur économique exploitable par le solveur.

5.1.2 Résolution du secteur

La fonction centrale est **solve_sector** :

```
def solve_sector(vertices, edges, arcs, required=None, backend="CBC"):
    """
    vertices : list of vertex ids
    edges    : TWO-WAY streets
    arcs     : ONE-WAY streets
    backend  : OR-Tools backend
    """
```

Cette fonction justifie le choix de l'optimisation exacte :

- les variables entières représentent le nombre de passages sur chaque arc ;
- les contraintes assurent la couverture des rues requises ;
- la conservation des flux assure un circuit fermé ;

- la fonction objectif minimise le coût total.

5.1.3 Reconstruction de tournée

Le projet utilise l'algorithme de Hierholzer :

```
def _hierholzer(remaining, start):
    """Eulerian circuit through the component reachable from start."""
```

Le projet inclut aussi :

```
def build_route_all(passes):
    """Ordered route covering EVERY driven arc."""
```

Cette approche est particulièrement utile lorsque le réseau prioritaire est constitué de plusieurs composantes déconnectées.

5.1.4 Conversion depuis OpenStreetMap

```
def osmnx_to_graph(G):
    """
    Convert an OSMnx MultiDiGraph into
    (vertices, edges, arcs, lengths).
    """
```

Cette fonction établit le lien entre la carte réelle et le modèle mathématique.

5.2 Le module priorities.py

Ce module définit trois scénarios de priorisation.

5.2.1 Scénario artériel

```
def arterial(G):
    """Tier 1 = grandes voies."""
```

Ce scénario privilégie les grands axes routiers.

5.2.2 Scénario services essentiels

```
def essential_services(
    G,
    place,
    dist=150.0,
    amenities=("hospital", "school", "fire_station", "clinic")
):
```

Ce scénario repose sur l'accessibilité aux équipements stratégiques.

5.2.3 Scénario transport collectif

```
def transit(G, place, dist=100.0):
```

Ce scénario favorise les rues proches des arrêts de bus.

5.3 Le module scenarios.py

5.3.1 Mesures de base

```
def route_metrics(passes, lengths):
    """(km parcourus, heures)"""
```

Ces fonctions convertissent la solution du solveur en indicateurs interprétables.

5.3.2 Couverture du réseau prioritaire dans le temps

```
def coverage_over_time(route, lengths, priority, times_h):
```

Cette fonction permet de mesurer à quelle vitesse un scénario rend le réseau prioritaire praticable.

5.3.3 Exécution d'un scénario

```
def run_scenario(place, scenario, n_vehicles=None, n_max=10):
```

Logique :

- chargement du secteur ;
- identification du réseau prioritaire ;
- résolution en deux phases ;
- calcul des indicateurs ;
- calcul du coût final.

5.3.4 Baseline et export

```
def run_baseline(place, n_vehicles=None, n_max=10):
```

La baseline est indispensable pour comparer objectivement les scénarios.

5.4 Le module `cost_model.py`

```
def daily_cost(total_km, total_hours, n_vehicles):
```

Ce module relie directement les tournées calculées à une interprétation économique.

5.5 Le script `demo.py`

```
def main():  
    print(f"Secteur : {PLACE}")
```

Ce script fournit une démonstration simple du projet.

5.6 L'interface `app.py`

```
col1, col2, col3 = st.columns(3)  
col1.metric("Coût total ($)", r["cout_total"])  
col2.metric("Km parcourus", r["km_total"])  
col3.metric("Heures totales", r["heures_total"])
```

Cette interface facilite la comparaison rapide des scénarios.

6. Choix du point de départ

Dans la version actuelle du projet, le point de départ n'est pas défini comme un dépôt réel de déneigeuses.

Ce choix a été fait afin de concentrer le modèle sur le problème principal de couverture optimale du réseau.

Il s'agit d'une simplification volontaire qui pourrait être améliorée à l'avenir en intégrant un dépôt réel comme contrainte logistique.

7. Indicateurs fournis par le projet

Le projet produit plusieurs indicateurs :

- coût total journalier ;
- distance totale parcourue ;
- temps total estimé ;
- part de kilomètres à vide ;
- temps de dégagement du réseau prioritaire ;
- couverture du réseau prioritaire après 1 h, 2 h et 4 h ;
- nombre minimal de véhicules pour terminer en moins de 8 heures.

8. Utilisation du projet

Installation

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install osmnx
```

Sous Windows :

```
.venv\Scripts\activate
```

Démonstration simple

```
python demo.py
```

Calcul complet des secteurs

```
python scenarios.py --all
```

Interface graphique

```
streamlit run app.py
```

Tests

```
python -m pytest
```

9. Tableaux par secteur

Les tableaux suivants présentent, pour chaque secteur, les performances de la solution de base et des trois scénarios de priorisation. Les indicateurs retenus sont la distance totale parcourue, le temps total estimé, le coût journalier correspondant ainsi que, pour les scénarios de priorisation, la durée de traitement complet du réseau prioritaire (T1).

Outremont

Scénario	Distance totale (km)	Temps total (h)	Coût total (\$)	T1 réseau prioritaire (h)
Baseline	53,10	5,31	564,28	—
Artériel	59,30	5,93	571,76	1,182
Services	59,00	5,90	571,40	2,443
Transport	59,20	5,92	571,65	3,619

Verdun

Scénario	Distance totale (km)	Temps total (h)	Coût total (\$)	T1 réseau prioritaire (h)
Baseline	83,20	8,32	1 100,69	—
Artériel	88,90	8,90	1 107,64	3,314
Services	93,90	9,39	1 113,59	3,847
Transport	87,90	8,79	1 106,38	7,859

Anjou

Scénario	Distance totale (km)	Temps total (h)	Coût total (\$)	T1 réseau prioritaire (h)
Baseline	196,40	19,64	1 737,60	—
Artériel	211,70	21,17	1 756,23	10,772
Services	210,20	21,02	1 754,36	3,210
Transport	234,40	23,44	1 783,65	14,322

Rivière-des-Prairies – Pointe-aux-Trembles

Scénario	Distance totale (km)	Temps total (h)	Coût total (\$)	T1 réseau prioritaire (h)
Baseline	540,00	54,00	4 153,37	—
Artériel	556,80	55,68	4 173,75	19,293
Services	558,50	55,85	4 175,79	7,654
Transport	592,40	59,24	4 716,72	32,977

10. Forces du projet

Le projet présente plusieurs qualités :

- une modélisation rigoureuse du réseau routier ;
- une prise en compte correcte des sens de circulation ;
- une méthode exacte d'optimisation ;
- une architecture modulaire claire ;
- plusieurs scénarios de priorisation ;
- un modèle de coût intégré ;
- des outils de visualisation et d'export.

11. Limites du projet

11.1 Dépendance aux données OpenStreetMap

Le résultat dépend de la qualité des données cartographiques disponibles.

11.2 Réduction à la plus grande composante fortement connexe

Cette étape améliore la faisabilité du solveur, mais peut exclure certaines rues.

11.3 Vitesse moyenne constante

Le modèle suppose une vitesse fixe de 10 km/h.

11.4 Répartition idéale du travail

Le modèle suppose une répartition idéale entre plusieurs véhicules.

11.5 Point de départ simplifié

Le dépôt réel n'est pas encore intégré.

11.6 Priorisation géométrique simplifiée

Les scénarios reposent sur des seuils fixes qui simplifient la réalité.

12. Conclusion

Le projet propose une solution cohérente et rigoureuse au problème du déblaiement routier dans plusieurs secteurs de Montréal.

En s'appuyant sur :

- une modélisation en graphe orienté ;
- une résolution exacte par programmation linéaire entière ;
- une reconstruction de tournée par l'algorithme de Hierholzer ;

il permet de produire des itinéraires réalistes et économiquement interprétables.

L'ajout de scénarios de priorisation constitue une contribution importante, car il permet d'évaluer différentes politiques de déneigement au-delà d'une simple logique de minimisation de distance.

Le modèle de coût, les exports JSON/CSV, la démonstration et l'interface Streamlit renforcent encore l'intérêt pratique du projet.

Le travail réalisé constitue ainsi une base solide pour l'analyse comparative des stratégies de déneigement, qui pourra être complétée par la suite par l'exploitation des résultats chiffrés.